VFS to provide the user-space file-system interface. However, he is yet to figure out how to read from the underlying block device in kernel space. Information on which block device to use is already embedded into the *super_block* structure itself, obtained from the user issuing the mount command. But, since Pugs decided to get a bare-bones real SFS up first, he went ahead and wrote his *sfs_super_fill()* function also as a hard-coded fill function. And with that, he registered the Simula file system with the VFS. As for any other Linux driver, here's the file system driver's constructor and destructor:

```
#include <linux/module.h> /* For module related macros, ... */

static int __init sfs_init(void) {
    int err;

    err = register_filesystem(&sfs);
    return err;
}

static void __exit sfs_exit(void) {
    unregister_filesystem(&sfs);
}

module_init(sfs_init);
module_exit(sfs_exit);
```

Both *register_filesystem()* and *unregister_filesystem()* take a pointer to the *struct file_system_type sfs* (filled above) as their parameter, to respectively register and unregister the file system described by it.

Hard-coded SFS superblock and root inode

And yes, here's the hard-coded *sfs_fill_super()* function:

```
#include "real_sfs_ds.h" /* For SFS related defines, data
structures, ... */

static int sfs_fill_super(struct super_block *sb, void *data, int
silent) {
    printk(KERN_INFO "sfs: sfs_fill_super\n");

    sb->s_blocksize = SIMULA_FS_BLOCK_SIZE;
    sb->s_blocksize_bits = SIMULA_FS_BLOCK_SIZE_BITS;
    sb->s_magic = SIMULA_FS_TYPE;
    sb->s_type = &sfs; // file_system_type
    sb->s_op = &sfs_sops; // super block operations

    sfs_root_inode = iget_locked(sb, 1); // obtain an inode from
VFS
    if (!sfs_root_inode) {
        return -EACCES;
    }
    if (sfs_root_inode->i_state & I_NEW) // allocated fresh now
    {
        printk(KERN_INFO "sfs: Got new root inode, let's fill
in\n");
        sfs_root_inode->i_op = &sfs_iops; // inode operations
        sfs_root_inode->i_mode = S_IFDIR | S_IRWXU |
            S_IRGRP | S_IXGRP | S_IROTH | S_IXOTH;
        sfs_root_inode->i_fop = &sfs_fops; // file operations
        sfs_root_inode->i_mapping->a_ops = &sfs_aops; // address
operations
        unlock_new_inode(sfs_root_inode);
    }
    else {
        printk(KERN_INFO "sfs: Got root inode from inode
cache\n");
    }

#if (LINUX_VERSION_CODE < KERNEL_VERSION(3,4,0))
    sb->s_root = d_alloc_root(sfs_root_inode);
#else
    sb->s_root = d_make_root(sfs_root_inode);
#endif
    if (!sb->s_root) {
        iget_failed(sfs_root_inode);
        return -ENOMEM;
    }

    return 0;
}
```

As mentioned earlier, this function is basically supposed to read the underlying SFS superblock and, accordingly, translate and fill the passed *struct super_block* (as pointed by the first parameter, *sb*). So understanding it is equal to understanding the minimal fields of the *super_block* struct, which are filled. The first three are: the block size, its logarithm Base 2, and the type/magic code of the Simula file system. As Pugs codes further, we will see that once he gets the superblock from the hardware, he would instead get these values from the superblock and, more importantly, verify them, to ensure that the correct partition is being mounted.

After that, the various structure pointers are pointed at the corresponding structures of function pointers. Last, but not least, the *root_inode* pointer *s_root* is pointed to the *struct inode* structure, obtained from VFS' inode cache, based on the inode number of *root*—which right now has been hard-coded to 1; it may possibly change. If the inode structure is obtained fresh, i.e., for the first time, it is filled as per the underlying content of SFS' root inode. Also, the *mode* field is being hard-coded to *"drwxr-xr-x"*. Apart from that, the usual structure pointers are being initialised by the corresponding structure addresses. And finally, the *root's* inode is being attached to the superblock using *d_alloc_root()* or *d_make_root*, as per the kernel version.

All the above code pieces have been put in together as